

Exact Family Patching versus Geometry-Key Canonicalization for Deduplicating DXF Drawings: A Pre-Registered Evaluation

Michael Cohee

Independent Researcher, New York, NY, USA

michaelacohee@gmail.com

Abstract

Computer-aided design (CAD) archives contain many drawings that differ only slightly, which suggests that a representation aware of geometry could deduplicate them better than byte-level methods. We tested that hypothesis for DXF drawings. A canonicalizing adapter (A3) maps drawing entities to polynomial geometry keys, uses the keys only as candidate lookups, and guarantees byte-exact restore from stored raw bytes plus verified edits. We compared it with fixed-chunk deduplication (A1), a cheap exact normalization (A2), content-defined chunking (C1) and per-family delta patching (C2). Every arm was stored through the same variable-final-share 4+2 Reed–Solomon layer and restored byte-for-byte under all 21 one- and two-share loss patterns. Rules were frozen at recorded commits before test data were generated, and each test set was opened once. On 44 development files, A3 v2 used 9.7% fewer protected bytes than A1, but on a held-out set of 132 files it used 8.1% more; C2 was smallest in both. A census of 47 real drawings found only 1.31% of eligible bytes to be candidates for pose-invariant matching, and the largest cluster consisted of two-vertex line segments rather than reused profiles. We therefore reject the canonicalizer as a storage format. Used only to choose the base file for C2 patching, the geometry key reduced protected bytes by 1.8% versus filename families on the 47 real drawings (0.8% on generated data) with zero verification failures. The result is a small, exact-restore-safe gain, not evidence that the approach generalizes.

Keywords: deduplication; delta encoding; CAD; DXF; canonicalization; negative result; pre-registration; erasure coding

1. Introduction

Engineering and architectural archives accumulate near-duplicate drawings: revisions of one detail, variants that differ in a dimension, and copies that are moved, rotated or scaled. Byte-level deduplication sees such files as mostly distinct. The hypothesis tested here is that a canonical, geometry-aware representation of each drawing would expose shared content that byte methods miss, and so reduce stored bytes.

The author originally framed this as polynomial canonicalization: many different expressions can denote the same value, so a store could keep one canonical form plus a control and discard the rest. Applied to files, that framing has a known risk. A canonical form preserves a value, not a file, and a storage system must return the original bytes. The design and evaluation below therefore separate a fuzzy lookup (which may be wrong) from a literal restore (which must be exact).

This report makes four contributions:

- A **DXF adapter** in which the geometry key proposes candidates and exactness comes only from stored raw bytes and verified edits (Section 3).
- A **pre-registered comparison** of six storage arms, with a protected-versus-derived byte ledger, frozen rules and single-use test sets (Section 4).

- A **negative result**: the canonicalizer's development gain reversed on held-out data, and a census found almost no pose-invariant targets in real drawings (Section 5).
- A **small positive result**: the geometry key helps when used only to pick the base file for delta patching (Section 5.4).

The erasure-coding layer is RedTail-X [1] and is held fixed; its behaviour is not re-evaluated here.

2. Background and Related Work

Chunk-level deduplication. Content-addressed stores split data into fixed-size blocks and store each distinct block once [2]. Content-defined chunking places boundaries by content so that insertions do not shift every later chunk [3]; FastCDC is a fast variant [4].

Delta encoding and base selection. Storing a file as a patch against a similar file is effective when the base is well chosen. Douglass and Iyengar showed that resemblance detection can select delta bases automatically [5], using sketches of the kind introduced by Broder [6]. The C2-key arm in Section 5.4 belongs to this line of work, with a domain-specific geometry key as the resemblance signal.

DXF. A DXF file is a stream of group-code/value pairs that encode header variables, tables, blocks and entities [7]. Posed copies of a shape are often stored inside one file as a BLOCK definition plus INSERT entities that carry position, scale and rotation.

Pre-registration. Fixing analysis rules before seeing test data reduces the risk of tuning a method to the data it is judged on [8]. We apply that practice to a systems benchmark by recording freeze commits and opening each test set once.

3. Design

3.1 Storage arms

Table 1. Storage arms. All arms restore every file byte-for-byte.

Arm	Representation
A1	Fixed-size chunks, exact deduplication by content hash.
A2	Cheap exact normalization, then A1-style deduplication: shortest-form number spelling, line-ending and whitespace cleanup, and volatile header fields (such as timestamps) moved into a counted undo record. No geometry is rewritten.
A3	Canonicalizing DXF adapter with polynomial geometry keys (v1 and v2; Section 3.2).
C1	Content-defined chunking (FastCDC [4]) with exact deduplication.
C2	Per-family delta patching: each file is stored whole or as a verified patch against an earlier file.

Each arm was run in mode N (no compression) and mode Z (Zstandard [9]); tables report mode Z. Output of every arm was striped through RedTail-X 4+2 with variable final shares [1].

3.2 The A3 adapter

Adapter v1 (frozen at commit `addac59`) reads raw DXF tags, passes unrecognized records through as raw bytes, and computes a polynomial geometry key per entity. The key is used only to look up candidates. An entity is shared only when the stored bytes, after applying any edits from a closed list, reproduce the original exactly; otherwise it is stored as new. On the development set all 44 files restored byte-for-byte, and the adapter and library test suites (7 and 25 tests) passed.

Adapter v2 (frozen at b0f355e) changed how references are stored: whole-file references first, then run-level references, and per-entity references only as a last resort. It also compressed the per-file edit stream, recorded number re-spelling as a named undo edit, and moved the key index out of protected storage (Section 4.1).

4. Method

4.1 Protected and derived bytes

The primary metric is protected bytes: everything required to restore every file after all derived data is deleted. That includes payloads, edit streams, references, per-stripe manifests and the catalog of stored objects. Derived bytes are structures that can be rebuilt from protected data, such as the geometry-key index; they are reported on a separate line. An index qualifies as derived only if it passes a rebuild test: delete it, restore all files, rebuild it, and obtain an identical verify-pass vector. The A3 v2 index passed this test.

4.2 Data

- **Development set:** 44 DXF files, including generated moved, rotated and scaled variants.
- **Held-out set** darkrock-cad-dxf-v2: 132 generated DXF files, created after the v2 freeze and opened once. An earlier 132-file split was discarded because an early A1 baseline had read it.
- **Real set:** 47 real architectural detail drawings, grouped into product families by filename (for example, family 083213: sliding aluminum-framed glass doors).
- **Base-selection test:** a further generated set (“v3”), created after the base-selection rules were frozen at c4525d0; its test half was opened once.

4.3 Correctness

For every arm and set, each stored file was restored and compared byte-for-byte after each of the 21 one- and two-share loss patterns of the 4+2 layer. Key lookups that failed verification were counted and never merged.

5. Results

5.1 Adapter v1 on development data

The matching logic worked: 34,469 key lookups, all 34,469 verified, zero unverified merges, and 0.18% of bytes passed through unrecognized. One edit type fired 9,894 times. Storage, however, failed badly. A3 v1 needed 15,765,756 protected bytes (11,785,044 once the key index was moved to the derived line) against 3,612,642 for A1 and 2,286,900 for C2. Per-entity references and the key index (about 5.6 MB) outweighed C2's entire total, and an exact second copy of one file cost A3 v1 613 KB against 84 bytes for A1. Adapter v1 was rejected on storage.

5.2 Adapter v2: development versus held-out

Table 2. Protected bytes, mode Z, variable-final-share 4+2. Percentages are relative to A1.

Arm	Development (44)	Held-out (132)	Change vs. A1 (dev → held-out)
C2 family patch	2,286,900	11,369,532	−36.7% → −26.3%
C1 FastCDC	3,150,030	13,712,514	−12.8% → −11.1%
A2 cheap normalize	3,354,036	14,555,964	−7.2% → −5.6%
A1 fixed-chunk	3,612,642	15,424,938	baseline
A3 v2 canonicalizer	3,260,730	16,670,892	−9.7% → +8.1%

On development data A3 v2 fixed v1's copy cost (an exact second copy cost 66 bytes, against 72–84 bytes for A1) and beat A1 by 9.7% and A2 by 2.8%, while still using 43% more than C2. The development gain did not generalize: on the held-out set A3 v2 used 8.1% more than A1. Per-entity references dominated on drawings with reordered entities or many small entities. Its derived index was 2,653,088 bytes on development data; on the held-out set, counting the index as protected would raise A3 v2 to 44,757,804 bytes.

A2, the exact cheap normalization, gave a consistent saving of 5.6–7.2% over A1. C2 was the smallest arm on both sets.

5.3 Census of pose-invariant targets

A planned v3 adapter would use a key invariant to translation, rotation and scale. Before building it, we ran a declared census on the 47 real drawings. Only 865,098 bytes (1.31% of eligible bytes) were posed-key candidates, before verification. Family 083213 held 749,871 of those bytes (86.7%), but a breakdown by entity size showed that 96% of its 4,266 candidate records were two-vertex LWPOLYLINE segments (median record 174 bytes, largest 329 bytes), almost all in two files. Under a similarity-invariant key every straight segment resembles every other, so these matches are key noise, not reused profiles. Even if all candidates were real and free, the saving would be about 0.5% of that set's C2 total. v3 was not built.

5.4 Geometry key as a delta-base selector

Because C2 won on storage, the final experiment used the geometry key only to choose which earlier file each file is patched against, with literal restore. Arms: no base (C2-0), filename family (C2-name), a generic similarity sketch (C2-sketch), geometry key (C2-key), and filename family with key fallback (C2-name+key).

Table 3. Protected bytes for C2 base selection. Rules frozen at c4525d0; all arms byte-exact under 21 loss patterns; zero patch verification failures.

Base selection	Generated (v3 test)	47 real drawings	Real vs. C2-0
C2-0 (no base)	18,685,896	23,139,486	—
C2-name	11,354,940	18,507,450	−20.0%
C2-sketch	11,952,642	18,254,496	−21.1%
C2-key	11,266,362	18,169,422	−21.5%
C2-name+key	11,354,940	18,299,088	−20.9%

Filename families provide most of the gain: 20% on real drawings. The geometry key adds 1.8% over filename families on real drawings (338,028 bytes) and 0.8% on generated data, and 0.5% over the generic sketch on real drawings (5.7% on generated data). The hybrid did not beat key-only. The harm rate, meaning accepted patches for which the key’s base produced a larger patch than the filename base, was zero. Key computation time and memory were not included in the ledger.

6. Discussion

Why the canonicalizer failed. Matching was never the problem; every lookup verified. The cost lay in bookkeeping. A format that shares data at entity level must store a reference for every entity, and on drawings with many small or reordered entities those references cost more than the duplicated bytes they replace. File-level and run-level sharing, as in C2 and A1, avoids that cost.

Why the development result reversed. The development set included generated posed variants that favour a geometry-aware representation. The held-out set, generated after the freeze, did not reward it in the same way. Had we tuned and reported on the development set alone, we would have published a 9.7% gain that does not hold.

What remains useful. As a resemblance signal for delta-base selection [5], the geometry key gives a small gain with no verification failures. That is a narrow, defensible use of the idea, and it keeps the key out of the restore path.

7. Limitations

- Small corpora (44, 132 and 47 files) on a single volume of one machine; the generated sets may not reflect real archives.
- The 47 real drawings come from one domain (architectural details) and are not redistributable in this report.
- The key’s CPU and memory costs were not measured; a 1.8% saving could be outweighed by them.
- How concentrated the real-drawing margin is across files has not yet been reported; if it sits in two or three drawings, it is a family effect rather than a general policy.

- C1 and C2 parameters and the generator for the synthetic sets are documented in the source repository rather than restated here.

8. Conclusion

Geometry-key canonicalization of DXF drawings matched entities reliably but did not reduce storage: a 9.7% development gain became an 8.1% loss on held-out data, and real drawings offered almost no pose-invariant targets. We reject it as a storage format. Per-family delta patching was the most compact arm throughout, and a cheap exact normalization gave a steady 5–7%. Restricted to choosing delta bases, the geometry key reduced protected bytes by 1.8% on real drawings with byte-exact restore. The main lesson is methodological: freezing rules and holding out test data turned a promising development number into a correct negative result.

Author Contributions and Use of AI Tools

The author conceived the polynomial canonicalization idea and its application to CAD deduplication, directed every experiment, supplied the real drawings, ran all benchmarks on his own hardware, and made the decisions to reject v1, freeze v2, run the census before building v3, and restrict the key to base selection. He verified all reported results.

AI assistants were used throughout: ChatGPT and its coding agent (OpenAI) implemented the adapter, storage arms and benchmark harness and ran them on the author’s machine at his direction; Grok (xAI) reviewed the evaluation methodology, including the protected-versus-derived ledger, the index rebuild test, the constraints that kept A2 a fair baseline, split hygiene, the census buckets and the harm-rate check; Claude (Anthropic) assisted with analysis and drafted the text of this report. No AI system is listed as an author. The author takes full responsibility for the content.

Data and Code Availability

Source code, frozen commits (addac59, b0f355e, c4525d0) and the generator for the synthetic sets: <https://github.com/michaelcohee/DarkRock>. The real drawings are not redistributed.

References

- [1]M. Cohee, “RedTail-X: Variable-Length Final Shares for 4+2 Reed–Solomon Erasure-Coded Storage,” Technical Report v1.0, Sept. 2026.
- [2]S. Quinlan and S. Dorward, “Venti: A new approach to archival storage,” in *Proc. USENIX FAST*, 2002.
- [3]A. Muthitacharoen, B. Chen and D. Mazières, “A low-bandwidth network file system,” in *Proc. ACM SOSR*, 2001.
- [4]W. Xia, H. Jiang, D. Feng, L. Tian, M. Fu and Y. Zhou, “FastCDC: A fast and efficient content-defined chunking approach for data deduplication,” in *Proc. USENIX ATC*, 2016.

[5]F. Douglass and A. Iyengar, “Application-specific delta-encoding via resemblance detection,” in *Proc. USENIX ATC*, 2003.

[6]A. Z. Broder, “On the resemblance and containment of documents,” in *Proc. Compression and Complexity of Sequences*, 1997.

[7]Autodesk, Inc., “DXF Reference,” AutoCAD documentation.

[8]B. A. Nosek, C. R. Ebersole, A. C. DeHaven and D. T. Mellor, “The preregistration revolution,” *Proc. Natl. Acad. Sci. USA*, vol. 115, no. 11, pp. 2600–2606, 2018.

[9]Y. Collet and M. Kucherawy, “Zstandard Compression and the ‘application/zstd’ Media Type,” RFC 8878, IETF, 2021; implementation by Meta Platforms, Inc., <https://github.com/facebook/zstd>